

Relatório do Projeto Sistema Acadêmico

Trabalho Prático 2 - Autenticação e Integração de Dados

Instituto Federal de Mato Grosso - Campus Campo Verde

Disciplina: Programa WEB I

Professor: Samuel Oliveira Silva Bianchi

Discente: Cleifer Silva Moreira Matrícula: _____

Curso: TADS Semestre: 3º

Data: 28/05/2026

Este relatório descreve o desenvolvimento e os ajustes feitos no projeto Sistema Acadêmico, com foco no que foi solicitado no TP2. A ideia principal foi sair de uma estrutura mais estática e deixar o sistema funcionando com autenticação, banco de dados MySQL, Entity Framework Core e controle de acesso nas páginas internas.

Também foi tomado cuidado para manter o projeto o mais próximo possível da versão original, alterando principalmente o que era necessário para funcionar no ambiente local e para bater com as regras do enunciado.

1. Objetivo do trabalho

O objetivo do projeto foi implementar uma aplicação ASP.NET Core funcional para controle acadêmico, usando banco de dados e autenticação. O sistema trabalha com alunos, professores, disciplinas, notas e usuários do sistema.

Na prática, o trabalho teve duas partes principais: primeiro deixar a aplicação conectada corretamente ao banco de dados AulaAcademico, e depois ajustar o fluxo de login para que as telas internas só pudessem ser acessadas por usuários autenticados.

- Implementar login e logout de usuários.
- Gravar os dados em um banco MySQL, sem depender de dados fixos no código.
- Usar Entity Framework Core para fazer o mapeamento entre classes e tabelas.
- Centralizar a string de conexão no arquivo appsettings.json.
- Manter as operações de CRUD concentradas nas classes de Repository.

2. Arquitetura adotada

A arquitetura usada segue o padrão MVC do ASP.NET Core. As Views ficam responsáveis pela tela, os Controllers recebem as ações do usuário e os Models representam as entidades do sistema. Para organizar melhor o acesso ao banco, foi usada uma camada de Repositories.

O projeto ficou dividido principalmente da seguinte forma:

Parte do projeto	Responsabilidade
Models	Contêm as classes Aluno, Professor, Disciplina, Nota, Usuario e Pessoa.
Views	Contêm as páginas Razor usadas para listar, cadastrar, editar e excluir dados.
Controllers	Recebem as requisições, validam o fluxo e chamam os repositórios.
Repositories	Fazem as operações de consulta, cadastro, alteração e exclusão no banco.
Data / ApplicationDbContext	Configura o Entity Framework Core e os DbSet's do sistema.
appsettings.json	Guarda a configuração de conexão com o banco MySQL.

Essa separação ajudou bastante porque cada parte ficou com uma função mais clara. Um exemplo disso foi a correção feita para que os controllers de Disciplina e Nota parassem de acessar o DbContext diretamente, deixando o CRUD nos respectivos repositórios.

3. Banco de dados e persistência

O banco utilizado foi o MySQL, com a base AulaAcademico. A conexão fica configurada no appsettings.json e é carregada no Program.cs pelo DefaultConnection. O Entity Framework Core foi usado para criar o mapeamento das entidades e trabalhar com as tabelas.

O ApplicationDbContext possui DbSet's para Alunos, Professores, Disciplinas, Notas e Usuarios. Além disso, o relacionamento entre aluno e disciplina é feito pelo próprio EF Core com uma tabela intermediária chamada AlunoDisciplina.

Tabela / entidade	Campos principais	Observação
Usuarios	Id, Email, SenhaHash	Usada no login. A senha fica salva como hash.
Alunos	Id, Nome, Sobrenome, CPF, Email, DataNascimento, Matricula, Curso	Representa o aluno cadastrado no sistema.
Professores	Id, Nome, Sobrenome, CPF, Email, DataNascimento, Siap, Area	Representa o professor e sua área.
Disciplinas	Id, Nome, CargaHoraria, Periodo, ProfessorId	Pode ter professor vinculado.
Notas	Id, AlunoId, DisciplinaId, Valor, Descricao	Liga uma nota a um aluno e uma disciplina.
AlunoDisciplina	AlunoId, DisciplinaId	Tabela de relacionamento muitos-para-muitos.

Durante os testes, o banco foi populado com alguns registros de exemplo para deixar o sistema mais completo e facilitar a navegação pelas telas. Isso ajudou principalmente nas telas que possuem seleção de aluno, professor ou disciplina.

4. Autenticação e controle de acesso

O fluxo de autenticação foi implementado usando um AccountController. Ele possui as ações de Login, Cadastrar e Logout. Quando o usuário faz login corretamente, o e-mail é gravado na sessão, e isso permite acessar as telas internas do sistema.

Para impedir acesso direto sem login, foi criado o filtro AutenticacaoSessionAttribute. Esse filtro verifica se existe um usuário salvo na sessão. Se não existir, o usuário é redirecionado para a tela de login.

As senhas não são salvas em texto puro. No cadastro, a senha é transformada em hash usando BCrypt. No login, a senha digitada é comparada com o hash salvo no banco. Isso atende a regra do enunciado sobre não armazenar senha plana.

- A primeira página apresenta a opção para entrar na área de autenticação.
- As telas internas, como Alunos, Professores, Disciplinas e Notas, exigem login.
- O menu também foi ajustado para não permitir que o usuário pule o login pelos botões do topo.
- O logout limpa a sessão e retorna para a área pública.

5. Repositories e CRUD

Uma exigência importante do PDF era que apenas as classes Repositories realizassem operações de CRUD no banco. Por isso, foi conferido o uso do DbContext nos controllers e ajustado onde ainda estava fora do padrão.

Depois dos ajustes, os controllers ficaram responsáveis por controlar o fluxo da tela e chamar os métodos dos repositórios. Já as operações como adicionar, listar, atualizar e excluir ficaram nos repositórios de Aluno, Professor, Disciplina, Nota e Usuario.

Repository	Uso principal
AlunoRepository	Listagem, cadastro, atualização e exclusão de alunos, incluindo vínculo com disciplinas.
ProfessorRepository	CRUD dos professores.
DisciplinaRepository	CRUD das disciplinas e consulta dos períodos já cadastrados.
NotaRepository	CRUD das notas e consultas auxiliares de alunos e disciplinas para os selects.
UsuarioRepository	Cadastro e busca de usuários para autenticação.

6. Decisões de implementação

- 1 Manter o MVC original: a estrutura que já existia foi preservada, para não mudar o projeto mais do que o necessário.
- 2 Usar Session para autenticação: como o trabalho pedia login e controle de acesso, a sessão foi uma solução simples e suficiente para este escopo.
- 3 Usar BCrypt nas senhas: foi escolhido por ser uma forma mais segura de armazenar senha do que texto puro.

- 4 Usar appsettings.json para conexão: a string de conexão ficou centralizada, sem espalhar configurações pelo código.
- 5 Trocar campos manuais por selects em alguns cadastros: isso melhora o uso do sistema e evita digitação errada em campos que dependem de outro cadastro.

7. Dificuldades encontradas

A principal dificuldade foi a conexão com o MySQL no computador local. Como o projeto veio de outro ambiente, ele dependia de um banco que não existia na máquina. Foi necessário usar a base AulaAcademico, corrigir a configuração local do MySQL e aplicar as migrations para criar as tabelas.

Outra dificuldade foi o controle de acesso. Em um primeiro momento dava para acessar algumas telas internas pelo menu sem passar pelo login. Isso foi corrigido ajustando o layout e protegendo os controllers com o filtro de autenticação.

Também foi observado que algumas telas ainda estavam fazendo CRUD direto pelo DbContext dentro do controller. Isso entrava em conflito com o enunciado, então essa parte foi revisada e movida para os repositórios.

8. Estratégias usadas para resolver

- Verificação do enunciado para transformar os requisitos em uma lista de conferência.
- Testes no localhost para confirmar login, acesso protegido e navegação nas telas internas.
- Uso das migrations do Entity Framework Core para montar a estrutura do banco.
- Revisão dos controllers para confirmar que eles não estavam mais fazendo CRUD direto no banco.
- População do banco com dados de exemplo para validar selects e relacionamentos.

9. Conferência com o enunciado

Item pedido no PDF	Como foi atendido	Situação
Primeira página com botão para autenticação	A Home apresenta entrada para a tela de login.	Atendido
Apenas usuários autenticados acessam demais páginas	Controllers internos usam AutenticacaoSessionAttribute.	Atendido
Login e logout	AccountController implementa login, cadastro e logout.	Atendido
Senha não salva de forma plana	Senha armazenada como hash usando BCrypt.	Atendido
Banco MySQL	Aplicação conectada ao banco AulaAcademico.	Atendido
Entity Framework Core	ApplicationDbContext, DbSets e migrations foram usados.	Atendido

Item pedido no PDF	Como foi atendido	Situa??o
Configura??o no appsettings.json	DefaultConnection centraliza a conex?o.	Atendido
Sem dados fixos no c?digo	Dados principais v?m do banco e os selects buscam op??es cadastradas.	Atendido
CRUD apenas nos Repositories	Controllers chamam repositories para opera??es de banco.	Atendido

10. Testes realizados

Foram feitos testes b?sicos para confirmar o funcionamento do sistema no localhost. O primeiro teste foi abrir a p?gina inicial. Depois foi testado o acesso direto a uma tela interna sem login, que redirecionou para o login. Em seguida foi feito login com um usu?rio cadastrado e as p?ginas internas abriram normalmente.

- P?gina inicial abrindo no localhost.
- Tentativa de acesso interno sem login redirecionando para Login.
- Login com usu?rio cadastrado no banco.
- Abertura das p?ginas Alunos, Professores, Disciplinas e Notas ap?s autentica??o.
- Compila??o do projeto sem erros ap?s os ajustes.

11. Observa??o sobre apoio no desenvolvimento

Durante o processo foram usadas consultas, testes e apoio de ferramenta de IA para organizar alguns ajustes e revisar o atendimento aos requisitos. Mesmo assim, as altera??es foram conferidas no projeto, testadas no ambiente local e adaptadas para a realidade do sistema desenvolvido.

O conte?do entregue foi revisado com base no c?digo em funcionamento, principalmente nas partes de autentica??o, banco de dados e reposit?rios.

12. Conclus?o

Com os ajustes realizados, o Sistema Acad?mico passou a atender o objetivo principal do TP2: funcionar como uma aplica??o ASP.NET Core integrada ao MySQL, com autentica??o e dados persistidos. O sistema permite trabalhar com alunos, professores, disciplinas e notas, protegendo o acesso ?s telas internas.

O ponto mais importante da corre??o foi deixar o projeto funcionando no ambiente local e alinhar o acesso ao banco com a regra de uso dos reposit?rios. Assim, o projeto ficou mais organizado e mais pr?ximo do que foi solicitado no enunciado.